

Member ID: _____

Time: _____

Rank: _____



JAVA PROGRAMMING

(340)

NATIONAL 2025

PRODUCTION:

Java National

_____ (540 points)

Industry Certification (50 minutes)

IT Specialist Java

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
2. Electronic devices will be monitored according to ACT standards.
3. Your name and/or school name should *not* appear on work you submit for grading.
4. Ensure that the files required to run your program are present and will execute.
5. Once you complete the problem listed below, upload all competition materials (including project files, etc.) to the National BPA Pre-submit System at presubmit.bpa.org.
6. Compress all files in a single archive (zip file) and use the following naming conventions for your documents:

PART ONE Instructions: Production

Once you have completed listed below, upload all competition materials (including project files, etc.) to the National BPA Pre-submit System at presubmit.bpa.org.

Compress all files in a single archive (zip file) and use the following naming convention for your documents:

JAVA-MemberID.zip

PART TWO Instructions: Industry Certification

This exam requires IT Specialist Java Certification. Please do not leave your seat or exit the room until you have completed the certification. After the certification has been administered, you are permitted to leave.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder. The project folder for you has already been provided: Java National
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.
5. You will need to use a local Java IDE to complete this exam.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter. It is HIGHLY recommended that you place all of the files into one folder.

The graders will *not* compile or alter your source code to correct for this. Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- There are two Java files provided in the contest folder (AIChatBotGenerator_National && ChatBotReporter). You will be programming in both files.
 - **AIChatBotGenerator_National.java:** this is a static class. The job of this class file is to read the text files, organize the information and create chatbots. It will also pass control to the ChatBotReporter to run the menu and the sorting features.
 - **ChatBotReporter.java:** this is a static class. The job of this class file is to run the UI that will allow the user to print, sort the chatbots based upon primary language, and find the highest/lowest complexities.
- There are two text files in your chatbot folder: ExperimentalLanguageBots.txt, and Supported_Languages.txt.
 - **ExperimentalLanguageBots.txt:** these records are organized line by line, with each field being separated by a comma (like a csv. file but instead it is a text file). These chatbots are complete and only need to have their records imported and then constructed into a chatbot object and stored. Every language type has the prefix “_”; this is done to quickly identify the experimental chatbots, compared to the normal chatbots that will be randomly created, and to also have them grouped together when being sorted ascending and descending order. The languages for these experimental chatbots are all programming languages.
 - **Supported_Languages.txt:** this is a linear list of only languages separated with commas. The program will be used to create the primary and secondary language of randomly create chatbots. These languages are all natural languages (human languages).
- The purpose of the program is to create a program that can do the following:
 - Import two different types of text files, and combine them into one list
 - Find the highest complexity and lowest complexity ratings for all the chatbots.

- Sort the records in ascending and descending order based upon the primary language
- The primary language, secondary language will be collectively known as the “Default Languages.”
- Each chatbot record once imported will be organized by primary language, secondary language, Token Rate, and Complexity.
- When the records are combined, there will be a total of 22 chatbots.
- A menu UI will allow the user to select what to print to the console. There are seven variations of the records printed to the console, plus a final option to exit the program. These are the menu options:

1. Print Chat_Bots
2. Most Complex Chat_Bots
3. Least Complex Chat_Bots
4. Complexity Range (HIGH to LOW)
5. Complexity Range (LOW to HIGH)
6. Primary Language Range (A to Z)
7. Primary Language Range (Z to A)
0. Exit

- Below is an image of the experimental chatbot vs. the randomly created chatbot. The experimental will have the underscore symbol “_” prefixed in front of its language name (“_Swift”). The randomly created will only have the name (“Russian”).

_Swift, _Java	Token Rate: 22	Complexity: 100.0
Russian, French	Token Rate: 11	Complexity: 8.9186

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is a goal of good code.

Commenting for Source Code Review (see the rubric):

- Certain sections of your code will be graded. These gradable blocks of code can range from creating data structures, method algorithms, exception handling, and class construction.
- The grading rubric contains a section called Source Code Review: in this section are listed a description of all of the graded programming concepts.
- Each gradable item must have a comment listed at its beginning, and the comment must be prefixed with the comment flag. The flag helps the graders easily locate the code to increase the effectiveness of grading.
- The flag will always use this naming convention: **SC#** (NOTE: the # symbol will be replaced with sequential numbering, i.e. **SC1, SC2, SC3**, etc.
- No explanation in the comment with the flag is required, only the comment flag; however, any information placed in the comment could help the grader better understand and avoid any costly errors.
- The comment flag needs to be placed in close proximity to the block of code it represents.
- If a comment flag is not present, you will not receive credit.
- In this example, the Source Code Review has a gradable section of code for printing to the console (remember these are non-related examples):
 - SC12: *print* method in the *main* class is printing the correct object ____ 10 pts
 - The user will place the code above the method call:

```
//SC12 printing the car object  
System.out.println(car);
```

Java_AIChatBotGenerator_National

The goal of this contest is to create a software application that will import data from text files and use this information to create new chatbots and giving the user a menu UI that will allow them to generate reports. The **AIChatBotGenerator_National.java** will be designed to import, read, and create chatbot records based upon two different types of text files. The **Supported_Languages.txt** file will be a list of languages only, that the program will create random chatbot records similar to the Java_State contest. The **ExperimentalLanguageBots.txt** file will be complete chatbot records that the program must use to create chatbot records. These chatbots will have computer programming languages for their default languages. All records should be stored into one data structure.

The **ChatBotReporter.java** will be the program that hosts the menu UI, and sort the records in ascending (A to Z) and descending (Z to A) order based upon the primary language of the default languages. See a sample of the menu UI to the right.

```
Chat Bot 1: Default Languages: _Python, _JavaScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 2: Default Languages: _Java, _Go | Token Rate: 22 | Complexity: 1.0
Chat Bot 3: Default Languages: _C#, _Ruby | Token Rate: 22 | Complexity: 1.0
Chat Bot 4: Default Languages: _C++, _Swift | Token Rate: 22 | Complexity: 100.0
Chat Bot 5: Default Languages: _Go, _Python | Token Rate: 22 | Complexity: 1.0
Chat Bot 6: Default Languages: _Ruby, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 7: Default Languages: _C++, _C# | Token Rate: 22 | Complexity: 100.0
Chat Bot 8: Default Languages: _C#, _C++ | Token Rate: 22 | Complexity: 100.0
Chat Bot 9: Default Languages: _TypeScript, _PHP | Token Rate: 22 | Complexity: 1.0
Chat Bot 10: Default Languages: _Ruby, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 11: Default Languages: _JavaScript, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 12: Default Languages: _Swift, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 13: Default Languages: Russian, French | Token Rate: 11 | Complexity: 8.9186
Chat Bot 14: Default Languages: Vietnamese, English | Token Rate: 19 | Complexity: 9.1114
Chat Bot 15: Default Languages: Arabic, Japanese | Token Rate: 11 | Complexity: 10.2086
Chat Bot 16: Default Languages: French, Korean | Token Rate: 17 | Complexity: 8.0156
Chat Bot 17: Default Languages: Arabic, Swedish | Token Rate: 18 | Complexity: 10.5444
Chat Bot 18: Default Languages: Portuguese, Vietnamese | Token Rate: 14 | Complexity: 9.6091
Chat Bot 19: Default Languages: Korean, Spanish | Token Rate: 12 | Complexity: 4.5515
Chat Bot 20: Default Languages: Chinese, Russian | Token Rate: 17 | Complexity: 6.1229
Chat Bot 21: Default Languages: Spanish, German | Token Rate: 15 | Complexity: 1.7452
Chat Bot 22: Default Languages: Turkish, Czech | Token Rate: 17 | Complexity: 6.2696
```

```
>>>>>>>>>Choose a print option: <<<<<<<<<<
1. Print Chat_Bots
2. Highest Complex Chat_Bots
3. Lowest Complex Chat_Bots
4. Complexity Range (HIGH to LOW)
5. Complexity Range (LOW to HIGH)
6. Primary Language Range (A to Z)
7. Primary Language Range (Z to A)
0. Exit
```

Menu UI Option 1 Print

Chat_Bots: This output displays the imported experimental chatbots and the randomly created chatbots. Your program must have the experimental chatbots listed before the random chatbots. See image to the left.

Menu UI Option 2 Highest Complex Chat_Bots: This output displays the random chatbot that has the highest complexity rating. NOTE: this is only for the random chatbots, you will need to exclude the experimental chatbots from this filtering action.

```
-----Highest Complexity-----
Highest Complexity: 10.4168 Hungarian, Dutch
-----
```

Menu UI Option 3

Least Complex Chat_Bots: This output displays the random chatbot that has the least complexity rating. NOTE: this is only for the random chatbots, you will need to exclude the experimental chatbots from this filtering action.

```
-----Lowest Complexity-----
Least Complexity: 1.8498 Hungarian, Turkish
-----
```

Output 4 Complexity High to Low Range: The chatbot records will be sorted from greatest to least based upon Complexity values (see image to the right).

Output 5 Complexity Low to High Range: The chatbot records will be sorted from least to greatest based upon Complexity values (see image below).

```
-----Complexity High to Low Range-----
Chat Bot 1: Default Languages: _C++, _Swift | Token Rate: 22 | Complexity: 100.0
Chat Bot 2: Default Languages: _Ruby, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 3: Default Languages: _C++, _C# | Token Rate: 22 | Complexity: 100.0
Chat Bot 4: Default Languages: _C#, _C++ | Token Rate: 22 | Complexity: 100.0
Chat Bot 5: Default Languages: _Swift, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 6: Default Languages: _Swedish, _Spanish | Token Rate: 15 | Complexity: 10.7726
Chat Bot 7: Default Languages: _Polish, _Hebrew | Token Rate: 12 | Complexity: 10.4104
Chat Bot 8: Default Languages: _Hungarian, _Italian | Token Rate: 18 | Complexity: 8.1061
Chat Bot 9: Default Languages: _Indonesian, _Spanish | Token Rate: 19 | Complexity: 6.3465
Chat Bot 10: Default Languages: _English, _Swedish | Token Rate: 12 | Complexity: 5.4275
Chat Bot 11: Default Languages: _Japanese, _Portuguese | Token Rate: 12 | Complexity: 4.8408
Chat Bot 12: Default Languages: _Indonesian, _Vietnamese | Token Rate: 14 | Complexity: 3.8251
Chat Bot 13: Default Languages: _Polish, _Thai | Token Rate: 20 | Complexity: 3.7026
Chat Bot 14: Default Languages: _Greek, _Arabic | Token Rate: 14 | Complexity: 2.477
Chat Bot 15: Default Languages: _Thai, _Vietnamese | Token Rate: 16 | Complexity: 2.2591
Chat Bot 16: Default Languages: _Python, _JavaScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 17: Default Languages: _Java, _Go | Token Rate: 22 | Complexity: 1.0
Chat Bot 18: Default Languages: _C#, _Ruby | Token Rate: 22 | Complexity: 1.0
Chat Bot 19: Default Languages: _Go, _Python | Token Rate: 22 | Complexity: 1.0
Chat Bot 20: Default Languages: _TypeScript, _PHP | Token Rate: 22 | Complexity: 1.0
Chat Bot 21: Default Languages: _Ruby, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 22: Default Languages: _JavaScript, _TypeScript | Token Rate: 22 | Complexity: 1.0
-----
```

```
-----Complexity Low to High Range-----
Chat Bot 1: Default Languages: _Python, _JavaScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 2: Default Languages: _Java, _Go | Token Rate: 22 | Complexity: 1.0
Chat Bot 3: Default Languages: _C#, _Ruby | Token Rate: 22 | Complexity: 1.0
Chat Bot 4: Default Languages: _Go, _Python | Token Rate: 22 | Complexity: 1.0
Chat Bot 5: Default Languages: _TypeScript, _PHP | Token Rate: 22 | Complexity: 1.0
Chat Bot 6: Default Languages: _Ruby, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 7: Default Languages: _JavaScript, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 8: Default Languages: _Thai, _Vietnamese | Token Rate: 16 | Complexity: 2.2591
Chat Bot 9: Default Languages: _Greek, _Arabic | Token Rate: 14 | Complexity: 2.477
Chat Bot 10: Default Languages: _Polish, _Thai | Token Rate: 20 | Complexity: 3.7026
Chat Bot 11: Default Languages: _Indonesian, _Vietnamese | Token Rate: 14 | Complexity: 3.8251
Chat Bot 12: Default Languages: _Japanese, _Portuguese | Token Rate: 12 | Complexity: 4.8408
Chat Bot 13: Default Languages: _English, _Swedish | Token Rate: 12 | Complexity: 5.4275
Chat Bot 14: Default Languages: _Indonesian, _Spanish | Token Rate: 19 | Complexity: 6.3465
Chat Bot 15: Default Languages: _Hungarian, _Italian | Token Rate: 18 | Complexity: 8.1061
Chat Bot 16: Default Languages: _Polish, _Hebrew | Token Rate: 12 | Complexity: 10.4104
Chat Bot 17: Default Languages: _Swedish, _Spanish | Token Rate: 15 | Complexity: 10.7726
Chat Bot 18: Default Languages: _C++, _Swift | Token Rate: 22 | Complexity: 100.0
Chat Bot 19: Default Languages: _Ruby, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 20: Default Languages: _C++, _C# | Token Rate: 22 | Complexity: 100.0
Chat Bot 21: Default Languages: _C#, _C++ | Token Rate: 22 | Complexity: 100.0
Chat Bot 22: Default Languages: _Swift, _Java | Token Rate: 22 | Complexity: 100.0
-----
```

Output 6 Primary Language Order (Low [A] to High Range [Z]): The chatbot records will be sorted from Low to High based upon the alphanumeric value (see image to the right). NOTE: the underscore symbol “_” has a greater value than “Z” which is why those records will appear at the end. See image below.

Output 7 Primary Language Order (High [Z] to Low Range [A]): The chatbot records will be sorted from High to Low based upon the alphanumeric value (see image to the right). NOTE: the underscore symbol “_” has a greater value than “Z” which is why those records will appear at the end. See image below.

```
-----Primary Language Order (Low [A] to High Range [Z])-----
Chat Bot 1: Default Languages: Dutch, Indonesian | Token Rate: 13 | Complexity: 5.316
Chat Bot 2: Default Languages: Greek, Turkish | Token Rate: 19 | Complexity: 9.5159
Chat Bot 3: Default Languages: Greek, Hebrew | Token Rate: 13 | Complexity: 10.4519
Chat Bot 4: Default Languages: Indonesian, Dutch | Token Rate: 19 | Complexity: 6.4957
Chat Bot 5: Default Languages: Indonesian, Czech | Token Rate: 11 | Complexity: 7.4493
Chat Bot 6: Default Languages: Korean, Chinese | Token Rate: 16 | Complexity: 5.0265
Chat Bot 7: Default Languages: Polish, Indonesian | Token Rate: 18 | Complexity: 8.8291
Chat Bot 8: Default Languages: Spanish, Vietnamese | Token Rate: 16 | Complexity: 6.2209
Chat Bot 9: Default Languages: Thai, German | Token Rate: 13 | Complexity: 9.9149
Chat Bot 10: Default Languages: Turkish, French | Token Rate: 19 | Complexity: 10.3013
Chat Bot 11: Default Languages: _C#, _Ruby | Token Rate: 22 | Complexity: 1.0
Chat Bot 12: Default Languages: _C#, _C++ | Token Rate: 22 | Complexity: 100.0
Chat Bot 13: Default Languages: _C++, _Swift | Token Rate: 22 | Complexity: 100.0
Chat Bot 14: Default Languages: _C++, _C# | Token Rate: 22 | Complexity: 100.0
Chat Bot 15: Default Languages: _Go, _Python | Token Rate: 22 | Complexity: 1.0
Chat Bot 16: Default Languages: _Java, _Go | Token Rate: 22 | Complexity: 1.0
Chat Bot 17: Default Languages: _JavaScript, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 18: Default Languages: _Python, _JavaScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 19: Default Languages: _Ruby, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 20: Default Languages: _Ruby, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 21: Default Languages: _Swift, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 22: Default Languages: _TypeScript, _PHP | Token Rate: 22 | Complexity: 1.0
-----
```

```
-----Primary Language Order (High [Z] to Low Range [A])-----
Chat Bot 1: Default Languages: _TypeScript, _PHP | Token Rate: 22 | Complexity: 1.0
Chat Bot 2: Default Languages: _Swift, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 3: Default Languages: _Ruby, _Java | Token Rate: 22 | Complexity: 100.0
Chat Bot 4: Default Languages: _Ruby, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 5: Default Languages: _Python, _JavaScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 6: Default Languages: _JavaScript, _TypeScript | Token Rate: 22 | Complexity: 1.0
Chat Bot 7: Default Languages: _Java, _Go | Token Rate: 22 | Complexity: 1.0
Chat Bot 8: Default Languages: _Go, _Python | Token Rate: 22 | Complexity: 1.0
Chat Bot 9: Default Languages: _C++, _Swift | Token Rate: 22 | Complexity: 100.0
Chat Bot 10: Default Languages: _C++, _C# | Token Rate: 22 | Complexity: 100.0
Chat Bot 11: Default Languages: _C#, _Ruby | Token Rate: 22 | Complexity: 1.0
Chat Bot 12: Default Languages: _C#, _C++ | Token Rate: 22 | Complexity: 100.0
Chat Bot 13: Default Languages: Turkish, French | Token Rate: 19 | Complexity: 10.3013
Chat Bot 14: Default Languages: Thai, German | Token Rate: 13 | Complexity: 9.9149
Chat Bot 15: Default Languages: Spanish, Vietnamese | Token Rate: 16 | Complexity: 6.2209
Chat Bot 16: Default Languages: Polish, Indonesian | Token Rate: 18 | Complexity: 8.8291
Chat Bot 17: Default Languages: Korean, Chinese | Token Rate: 16 | Complexity: 5.0265
Chat Bot 18: Default Languages: Indonesian, Dutch | Token Rate: 19 | Complexity: 6.4957
Chat Bot 19: Default Languages: Indonesian, Czech | Token Rate: 11 | Complexity: 7.4493
Chat Bot 20: Default Languages: Greek, Turkish | Token Rate: 19 | Complexity: 9.5159
Chat Bot 21: Default Languages: Greek, Hebrew | Token Rate: 13 | Complexity: 10.4519
Chat Bot 22: Default Languages: Dutch, Indonesian | Token Rate: 13 | Complexity: 5.316
-----
```

Output 8 Exit the Program:
When “0” is selected the program will exit.

```
0. Exit
0
Exiting...
```

class ChatBotReporter Methods List

1. ***chatBotReporter(List<Chat_Bots> chat_Bots)***: This method is the main method of the class. It presents a menu to the user and allows them to choose different options for printing chat bot information based on complexity, language, and other criteria.
2. ***consoleMostComplex(List<Chat_Bots> chat_Bots)***: This method prints the chat bot with the highest complexity along with its details such as default languages and token rate.
3. ***consoleLeastComplex(List<Chat_Bots> chat_Bots)***: This method prints the chat bot with the lowest complexity along with its details.
4. ***consoleComplexRange_HIGH_to_low(List<Chat_Bots> chat_Bots)***: This method prints the chat bots in descending order of complexity, from highest to lowest.
5. ***consoleComplexRange_LOW_to_high(List<Chat_Bots> chat_Bots)***: This method prints the chat bots in ascending order of complexity, from lowest to highest.
6. ***consoleLanguageRange_LOW_to_high(List<Chat_Bots> chat_Bots)***: This method prints the chat bots in alphabetical order of secondary language, from A to Z.
7. ***consoleLanguageRange_HIGH_to_low(List<Chat_Bots> chat_Bots)***: This method prints the chat bots in reverse alphabetical order of secondary language, from Z to A.
8. ***printChat_Bots(List<Chat_Bots> chat_Bots)***: This method prints the details of all chat bots in the list, including default languages, token rate, and complexity. It is used by several other methods to print chat bot information.

class AIChatBotGenerator_National Methods List

1. ***main(String args[])***: This is the main method of the class where the program execution begins. It initializes and reads data from files, creates chat bot objects, and interacts with the user to perform various operations.
2. ***setChat_Bots(int cB)***: This method sets up the chat bots by generating random chat bot objects with specified attributes such as primary and secondary languages, token rate, and complexity.
3. ***readChatBotPremadeFrameworks(BufferedReader in)***: This method reads chat bot data from a file using a BufferedReader and constructs chat bot objects from the read data.
4. ***readChatBotLanguages() throws IOException***: This method reads supported languages from a file using a Scanner and populates an ArrayList<String> with the read languages. It may throw an IOException if there's an error reading the file.
5. ***getReader(String language)***: This method creates a BufferedReader to read data from a file specified by the language parameter. It handles FileNotFoundException and returns the BufferedReader.

Solution and Project (There is NO partial credit) (NOTE: UC represents uppercase and LC represents lowercase)		
The Java source file is present on the flash drive in a single folder with your contest ID		10 points
Program Execution (If the program does not execute, then the remaining items in this section receive a score of zero)		
Menu appears when program starts		10 points
Menu UI Option 1 Print Chat_Bots: first 12 records are from ExperimentalLanguageBots.txt , last 10 are random from Supported_Languages.txt		30 points
Menu UI Option 1 Print Chat_Bots: Output format matches the sample provided		10 points
Menu UI Option 2 Highest Complex Chat_Bots: displays the Highest Complexity		20 points
Menu UI Option 2 Highest Complex Chat_Bots: Output format matches the sample provided		10 points
Menu UI Option 3 Least Complex Chat_Bots: displays the Least Complexity		20 points
Menu UI Option 3 Least Complex Chat_Bots: Output format matches the sample provided		10 points
Output 4 Complexity High to Low Range displays records in the proper order		20 points
Output 4 Complexity High to Low Range format matches the sample provided		10 points
Output 5 Complexity Low to High Range displays records in the proper order		20 points
Output 5 Complexity Low to High Range format matches the sample provided		10 points
Output 6 Primary Language Order (Low [A] to High Range [Z]) displays records in the proper order		20 points
Output 6 Primary Language Order (Low [A] to High Range [Z]) format matches the sample provided		10 points
Output 7 Primary Language Order (High [Z] to Low Range [A]) displays records in the proper order		20 points
Output 7 Primary Language Order (High [Z] to Low Range [A]) format matches the sample provided		10 points
Subtotal		/240 Points

Source Code Review No credit will be given for missing flagged comments. Code segments must work to receive full credit.		
A comment containing the contestant number is present at the top of the AiChatBotGenerator_National.java file		10 points
SC1: Flag Comment Present		5 points
SC1: AiChatBotGenerator_National.java class: comment data structures declaration and assignment statement of information gathered from ExperimentalLanguageBots.txt .		10 points
SC2: Flag Comment Present		5 points
SC2: AiChatBotGenerator_National.java class: comment data structures declaration and assignment statement of information gathered from Supported_Languages.txt .		10 points
SC3: Flag Comment Present		5 points
SC3: AiChatBotGenerator_National.java class: comment code file reading strategy was used for ExperimentalLanguageBots.txt using BufferedReader & readChatBotPremadeFrameworks()		15 points
SC4: Flag Comment Present		5 points
SC4: AiChatBotGenerator_National.java class readChatBotPremadeFrameworks() method: comment code demonstrating this method using BufferedReader readLine() method		15 points
SC5 A-C: Flag Comment Present		5 points
SC5 A-C: AiChatBotGenerator_National.java class readChatBotPremadeFrameworks() method: comment code //SC5A demonstrating finding delimiter (,) and //SC5B numerical data for integers and //SC5C for doubles from ExperimentalLanguageBots.txt		25 points
SC6: Flag Comment Present		5 points
SC6: AiChatBotGenerator_National.java class: comment code demonstrating ExperimentalLanguageBots.txt file NOT being found using try catch with exception being used		15 points
SC7: Flag Comment Present		5 points
SC7: AiChatBotGenerator_National.java class: comment code demonstrating file reading strategy was used for Supported_Languages.txt		15 points
SC8: Flag Comment Present		5 points
SC8: AiChatBotGenerator_National.java class readChatBotLanguages() method: comment code demonstrating gathering data from Supported_Languages.txt		15 points
SC9: Flag Comment Present		5 points
SC9: AiChatBotGenerator_National.java class readChatBotLanguages() method: comment code demonstrating removal of empty spaces from Supported_Languages.txt		15 points
SC10: Flag Comment Present		5 points
SC10: AiChatBotGenerator_National.java class: comment code with explanation of how to handle Supported_Languages.txt file NOT being found using try catch with an exception		15 points
SC11 A-B: Flag Comment Present		5 points
SC11 A-B: ChatBotReporter.java class method consoleMostComplex() comment code showing method was used by //SC11A menu and //SC11B implemented		10 points
SC12 A-B: Flag Comment Present		5 points
SC12 A-B: ChatBotReporter.java class method consoleLeastComplex() comment code showing method was used //SC12A menu and //SC12B implemented		10 points

SC13 A-B: Flag Comment Present		5 points
SC13 A-B: ChatBotReporter.java class method <i>consoleComplexRange_HIGH_to_low ()</i> comment code showing method was used //SC13A menu and //SC13B implemented		10 points
SC14 A-B: Flag Comment Present		5 points
SC14 A-B: ChatBotReporter.java class method <i>consoleComplexRange_LOW_to_high ()</i> comment code showing method was used //SC14A menu and //SC14B implemented		10 points
SC15 A-B: Flag Comment Present		5 points
SC15 A-B: ChatBotReporter.java class method <i>consoleLanguageRange_LOW_to_high ()</i> comment code showing method was used //SC15A menu and //SC15B implemented		10 points
SC16 A-B: Flag Comment Present		5 points
SC16 A-B: ChatBotReporter.java class method <i>consoleLanguageRange_HIGH_to_low ()</i> comment code showing method was used //SC16A menu and //SC16B implemented		10 points
Subtotal		/300 Points
Total Points		/540 Points